Техническое задание

на разработку универсальной мини-CRM «Custom-CRM»

Версия документа	1.0
Дата	12.11.2026
Статус	Утверждено
Заказчик	Локальный бизнес / владелец проекта
Исполнитель	AI-разработка (OpenCode)
Репозиторий	https://github.com/markmus11012020-max/custom-CRM.git

Документ подготовлен на основе шаблона технического задания и расширенного набора требований заказчика. Все перечисленные функции входят в согласованный объём проекта.

Содержание

Содержание 2

1. Общие сведения 5

1.1. Наименование и обозначение ПО . . . 5

1.2. Назначение ПО . . . 5

1.3. Область применения . . . 5

1.4. Термины и сокращения . . . 5

1.5. Роли пользователей . . . 5

1.6. Режимы эксплуатации . . . 5

1.7. Ограничения . . . 6

2. Цели и назначение 6

2.1. Цели . . . 6

2.2. Задачи . . . 6

2.3. Критерии успеха . . . 6

3. Функциональные требования 6

3.1. Аутентификация, пользователи и роли . . . 6

3.2. Проекты и мультипроектность . . . 7

3.3. Клиенты . . . 7

3.4. Сделки и заказы . . . 7

3.5. Задачи и напоминания . . . 7

3.6. Платежи и Robokassa . . . 7

3.7. Склад и позиции заказов . . . 8

3.8. Управленческий учёт . . . 8

3.9. Отчёты, выгрузка и расписание . . . 8

3.10. Уведомления . . . 8

3.11. Google Drive, Docs, Sheets . . . 8

3.12. Поиск . . . 9

3.13. API . . . 9

3.14. Дашборд . . . 9

3.15. Администрирование и настройки . . . 9

3.16. Импорт и экспорт . . . 9

4. Нефункциональные требования 9

4.1. Надёжность . . . 9

4.2. Производительность . . . 10

4.3. Масштабируемость . . . 10

4.4. Безопасность . . . 10

4.5. Совместимость . . . 10

4.6. Локализация . . . 10
4.7. Документирование . . . 10
4.8. Тестирование . . . 11
4.9. Логирование и мониторинг . . . 11
4.10. Резервное копирование . . . 11
5. Архитектура **11**
5.1. Общая схема . . . 11
5.2. Стек . . . 11
5.3. Структура проекта . . . 11
5.4. Потоки данных . . . 12
5.5. Аутентификация и права . . . 12
5.6. Фоновые задачи . . . 12
6. Модель данных **12**
6.1. Общие принципы . . . 12
6.2. Основные таблицы . . . 12
6.3. Статусы и этапы . . . 13
6.4. Индексы и ограничения . . . 13
7. API **13**
7.1. Общие принципы . . . 14
7.2. Аутентификация . . . 14
7.3. Основные endpoints . . . 14
8. Развёртывание **14**
8.1. Варианты запуска . . . 14
8.2. Docker и hot reload . . . 15
8.3. Переменные окружения . . . 15
8.4. Google Cloud . . . 15
8.5. Robokassa . . . 15
8.6. Уведомления . . . 15
8.7. Резервное копирование . . . 15
9. Тестирование и приёмка **16**
9.1. Виды тестов . . . 16
9.2. Чек-лист приёмки . . . 16
9.3. Definition of Done . . . 16
10. Этапы и сроки **16**
11. Ограничения и допущения **17**
12. Риски **17**
13. Приложения **17**

13.1. Переменные окружения 17
13.2. Примеры cron 18
13.3. Репозиторий 18

1. Общие сведения

1.1. Наименование и обозначение ПО

Полное наименование: «Универсальная мини-CRM Custom-CRM». Краткое наименование: Custom-CRM. Условное обозначение: CRM. Репозиторий проекта: https://github.com/markmus11012020-max/custom-CRM.git.

1.2. Назначение ПО

Custom-CRM — локальное или серверное веб-приложение для управления клиентами, проектами, сделками, заказами, задачами, платежами, складскими остатками и управленческим учётом. Система объединяет операционную работу менеджеров и аналитику руководителя в одном интерфейсе.

1.3. Область применения

- малый и средний бизнес, агентства, консалтинг, юридические и сервисные компании;
- личный бизнес владельца и команда до 20 пользователей;
- локальная установка на Windows с SQLite или серверная установка с PostgreSQL;
- работа через браузер, без установки клиентского приложения.

1.4. Термины и сокращения

Термин	Определение
Проект	Изолированная рабочая область с клиентами, сделками, задачами, платежами и отчётами.
Клиент	Физическое или юридическое лицо, с которым ведётся взаимодействие.
Сделка	Потенциальная или состоявшаяся продажа, привязанная к клиенту или проекту.
Заказ	Сделка с позициями и последующим списанием товаров со склада.
Задача	Действие или напоминание, связанное с клиентом, сделкой или проектом.
Платёж	Счёт или транзакция оплаты, в том числе через Robokassa.
Складская операция	Приход, расход или корректировка остатка товара.
Управленческая транзакция	Доход или расход проекта для расчёта прибыли.
Отчёт	Сводка данных CRM в Google Таблице, CSV или PDF-представлении.
Расписание	Cron-правило автоматической генерации отчёта.
OAuth2	Авторизация Google от имени личного аккаунта пользователя.
RBAC	Ролевое управление доступом (администратор, менеджер, наблюдатель).

1.5. Роли пользователей

Роль	Права
Администратор	Полный доступ: пользователи, проекты, настройки, интеграции, все данные, резервные копии.
Менеджер	CRUD клиентов, сделок, задач, платежей, склада и транзакций в проектах, где он участник.
Наблюдатель	Только чтение данных проектов, где он участник; экспорт отчётов.

1.6. Режимы эксплуатации

- Локальный режим: Windows 10/11, Docker Desktop, SQLite, запуск через start.bat.
- Серверный режим: Linux, Docker Compose, PostgreSQL, доступ через браузер.

• Режим разработки: монтирование кода в контейнер, Uvicorn reload, hot reload шаблонов и статики.

1.7. Ограничения

• Custom-CRM является инструментом управленческого учёта, а не заменой 1С или бухгалтерской программе.

• Robokassa подключается в тестовом режиме; для реальных платежей требуются боевые merchant-реквизиты и публичный ResultURL.

• Google-интеграция требует включённых API, согласия OAuth2 и доступа сервисного аккаунта к папке и таблице.

• Планировщик рассчитан на один экземпляр приложения; при горизонтальном масштабировании требуется внешний планировщик или блокировка.

• Мобильное приложение и офлайн-режим в объём не входят.

2. Цели и назначение

2.1. Цели

• сократить время на ведение клиентов и сделок за счёт единого интерфейса;

• обеспечить прозрачную воронку продаж и контроль задач;

• связать продажи, платежи, склад и управленческие финансы;

• автоматизировать подготовку отчётов и уведомлений;

• дать универсальную основу, которую можно развернуть локально или на сервере.

2.2. Задачи

• реализовать многопроектную CRM с ролевым доступом;

• реализовать CRUD для клиентов, сделок, заказов, задач и платежей;

• реализовать складской учёт и позиции заказов;

• реализовать управленческий учёт доходов и расходов;

• реализовать поиск, фильтры, пагинацию и экспорт;

• реализовать Google Drive, Google Docs и Google Sheets;

• реализовать Robokassa, email и Telegram-уведомления;

• реализовать планировщик отчётов и напоминаний;

• обеспечить запуск через Docker и start.bat.

2.3. Критерии успеха

• пользователь входит в систему и видит только доступные проекты;

• менеджер ведёт клиента от первого контакта до оплаты и выполнения заказа;

• руководитель видит воронку, финансы, склад и задачи без ручной сверки;

• отчёт формируется в Google Таблице одной кнопкой и по расписанию;

• уведомление приходит на email или в Telegram;

• проект запускается одной командой и обновляется без пересборки контейнера.

3. Функциональные требования

3.1. Аутентификация, пользователи и роли

• Система должна поддерживать вход по email и паролю, выход и смену пароля.

• Пароли должны храниться только в виде хеша Argon2; открытый пароль не сохраняется и не логируется.

• Сессия должна храниться в подписанной HttpOnly cookie; для API должен поддерживаться Bearer-токен.
• Администратор должен создавать, блокировать и редактировать пользователей, назначать роли.
• Должны поддерживаться роли admin, manager, viewer и проверка прав на каждом маршруте.
• Должны поддерживаться персональные API-токены с датой истечения и отзывом.
• Все изменения критичных сущностей должны попадать в журнал активности.

3.2. Проекты и мультипроектность

• Система должна поддерживать несколько проектов с названием, кодом, описанием, валютой и статусом.
• Пользователь должен переключать активный проект; все списки и KPI фильтруются по проекту.
• Администратор видит все проекты; менеджер и наблюдатель — только проекты, где они участники.
• Должны поддерживаться участники проекта и их роли.
• Проект должен архивироваться без удаления связанных данных.
• Удаление проекта должно требовать подтверждения и быть недоступно при наличии связанных записей, если не выбрано каскадное удаление.

3.3. Клиенты

• CRUD клиентов: имя, компания, email, телефон, статус, источник, ответственный, заметки.
• Статусы клиента: новый, активный, неактивный, архив.
• Мягкое архивирование и восстановление; жёсткое удаление только администратором.
• Карточка клиента: сделки, задачи, платежи, Google-файлы, история активности.
• Поиск по имени, компании, email, телефону и заметкам без учёта регистра.
• Фильтры по статусу, ответственному и архиву; пагинация.

3.4. Сделки и заказы

• CRUD сделок: название, клиент (необязательно), проект, тип (сделка/заказ), сумма, валюта, этап, описание.
• Этапы: новая, переговоры, предложение, выиграна, проиграна, выполнена.
• Сделка может существовать без клиента; клиент привязывается и отвязывается.
• Заказ должен поддерживать позиции: товар, наименование, количество, цена, сумма.
• При изменении позиций сумма заказа пересчитывается.
• Выполнение заказа должно списывать товары со склада и фиксировать движения.
• Смена этапа должна логироваться и при необходимости уведомлять ответственного.

3.5. Задачи и напоминания

• CRUD задач: заголовок, описание, клиент, сделка, проект, исполнитель, дедлайн, приоритет.
• Статусы: открыта, выполнена; поддержка повторного открытия.
• Фильтры: просроченные, сегодня, будущие, без срока, выполненные.
• Уведомление исполнителю при назначении и приближении дедлайна.
• Планировщик должен ежедневно проверять просроченные и близкие задачи.
• Задача может быть связана с клиентом, сделкой или только с проектом.

3.6. Платежи и Robokassa

• CRUD платежей: проект, клиент, сделка, сумма, валюта, назначение, статус, способ.
• Статусы: ожидает, оплачен, ошибка, возврат, отменён.

- Генерация ссылки Robokassa с корректной подписью MD5 и тестовым режимом IsTest=1.
- Проверка подписи ResultURL с Password2 и идемпотентное обновление статуса.
- Ручная отметка оплаты для локального режима.
- При оплате создаётся доходная управленческая транзакция и уведомление.
- При возврате создаётся компенсирующая расходная транзакция.
- Ссылка на оплату и история статусов отображаются в карточке платежа.

3.7. Склад и позиции заказов

- CRUD товаров: SKU, название, единица, цена продажи, себестоимость, остаток, минимальный остаток.
- Складские операции: приход, расход, корректировка; история движений.
- Контроль отрицательного остатка; запрет списания больше доступного количества.
- Расчёт стоимости склада по себестоимости.
- Список товаров с остатком ниже минимального.
- Позиции заказа связываются с товаром или вводятся вручную.
- Выполнение заказа создаёт расходные движения по позициям.

3.8. Управленческий учёт

- CRUD доходов и расходов: проект, тип, категория, сумма, дата, описание, связь с платежом/сделкой.
- Автоматическое создание дохода при оплате платежа.
- Расчёт выручки, расходов и прибыли за период и по проекту.
- Фильтры по периоду, типу, категории и проекту.
- Экспорт транзакций в CSV и Google Sheets.
- Подтверждение: система не формирует налоговую и бухгалтерскую отчётность.

3.9. Отчёты, выгрузка и расписание

- Типы отчётов: сводка CRM, продажи, финансы, склад, задачи.
- Генерация отчёта в Google Таблице с оформлением: заголовок, KPI, таблицы, итоги, диаграмма.
- AI-резюме через AITunnel; при недоступности — резервный текст без прерывания отчёта.
- CSV-выгрузка клиентов, сделок, задач, платежей, транзакций и склада.
- История отчётов со ссылками и статусом.
- Расписание: cron-выражение, проект, тип отчёта, получатели, каналы email/Telegram.
- Ручной запуск расписания и отображение следующего запуска.
- Ошибки генерации логируются и не блокируют следующие запуски.

3.10. Уведомления

- Email через SMTP и Telegram через Bot API.
- Настройки пользователя: email, Telegram chat_id, включение каналов.
- События: назначение задачи, приближение дедлайна, смена этапа сделки, оплата, отчёт, тестовое сообщение.
- Надёжная доставка через outbox-таблицу: pending, sent, failed, skipped.
- Повторные попытки и журнал ошибок без раскрытия паролей и токенов.
- Администратор видит статус интеграций и может отправить тестовое сообщение.

3.11. Google Drive, Docs, Sheets

- Сервисный аккаунт: чтение списка файлов, метаданных, скачивание, создание папок, загрузка, перемещение, переименование, удаление в корзину.

• OAuth2 личного аккаунта: вход, сохранение и обновление токена, создание Google Документов и Таблиц в заданной папке.
• Bootstrap: создание папки custom-CRM и выдача доступа сервисному аккаунту.
• Ссылки на созданные файлы сохраняются в CRM и отображаются в карточках.
• Ошибки Google API обрабатываются с понятными сообщениями; секреты не логируются.
• Точки входа: python -m app.google.service_account --list, python -m app.google.oauth_user --login, --create-test.

3.12. Поиск

• Поиск по клиентам, сделкам и задачам в активном проекте.
• Регистронезависимый поиск, включая кириллицу.
• Фильтры по статусу, этапу, исполнителю, датам и архиву.
• Пагинация и сортировка.
• API-поиск и поиск в интерфейсе используют единый сервис.

3.13. API

• REST API с JSON, версионированием /api и OpenAPI-документацией /docs.
• Аутентификация через сессию или Bearer-токен.
• Ролевые ограничения и фильтрация по проектам.
• Единый формат ошибок, коды 400/401/403/404/409/422/500.
• Пагинация, поиск, сортировка и фильтры.

3.14. Дашборд

• KPI: клиенты, активные сделки, сумма воронки, выручка, расходы, прибыль.
• Воронка по этапам и задачи по срокам.
• Последние сделки, платежи и события.
• Товары с низким остатком.
• Быстрые действия: создать клиента, сделку, задачу, платёж.

3.15. Администрирование и настройки

• Управление пользователями, ролями и проектами.
• Настройки Google, Robokassa, SMTP и Telegram.
• Проверка подключений и тестовые отправки.
• Просмотр журнала активности и системных логов.
• Резервное копирование и восстановление.

3.16. Импорт и экспорт

• Экспорт CSV с UTF-8 BOM для Excel.
• Экспорт в Google Sheets.
• Импорт клиентов и товаров из CSV — после MVP, архитектурно предусмотрен.
• Массовые операции и выгрузка отчётов — через API и UI.

4. Нефункциональные требования

4.1. Надёжность

• Все операции с БД выполняются в транзакциях.

- Платежи и уведомления идемпотентны.
- Healthcheck /health проверяет приложение и БД.
- Ошибки интеграций не приводят к потере данных.
- Резервное копирование SQLite и PostgreSQL описано и автоматизировано командой.
- Время восстановления после сбоя — до 30 минут при наличии резервной копии.

4.2. Производительность

- Время отклика типовой страницы — не более 2 секунд при локальной БД до 100 000 записей.
- Пагинация и индексы для списков.
- Фоновые отчёты и уведомления не блокируют HTTP-запросы.
- Кэширование тяжёлых агрегатов — при необходимости, без устаревания критичных данных.

4.3. Масштабируемость

- SQLite по умолчанию; PostgreSQL через DATABASE_URL.
- Docker-образ и конфигурация без привязки к Windows.
- API-first архитектура допускает замену Jinja2-интерфейса на React.
- Планировщик рассчитан на один экземпляр; для нескольких экземпляров — внешний scheduler.

4.4. Безопасность

- Argon2 для паролей, подписанные HttpOnly/SameSite cookie.
- CSRF-защита форм, параметризованные SQL-запросы.
- RBAC и проверка принадлежности к проекту.
- Секреты только в .env и secrets/, исключены из git.
- Логи без паролей, токенов и персональных секретов.
- Рекомендуется HTTPS reverse proxy при доступе вне localhost.

4.5. Совместимость

- Windows 10/11 и Linux.
- Python 3.12.
- Docker Engine 24+ и Docker Compose v2+.
- Современные браузеры Chrome, Edge, Firefox, Safari.
- Экран от 1280 px; адаптивная верстка для планшетов.

4.6. Локализация

- Русский интерфейс и сообщения об ошибках.
- Валюта RUB по умолчанию, поддержка иной валюты проекта.
- Часовой пояс Europe/Moscow по умолчанию, настраивается.
- Форматы дат ДД.ММ.ГГГГ и сумм с разделителями.

4.7. Документирование

- README без Markdown-разметки, обновляется после каждого изменения.
- Техническое задание в PDF.
- Руководство по Google Cloud и OAuth2.
- Руководство администратора и инструкция по устранению неполадок.

4.8. Тестирование

- pytest для БД, API, прав, поиска, платежей, склада, учёта и уведомлений.
- Моки Google API и SMTP/Telegram в автотестах.
- Ручной чек-лист UI и Docker.
- Минимальное покрытие критичных сервисов — 70%.

4.9. Логирование и мониторинг

- Ротация logs/app.log.
- Уровни DEBUG/INFO/WARNING/ERROR.
- Healthcheck и статус интеграций.
- Журнал активности пользователей.

4.10. Резервное копирование

- SQLite: копия файла data/crm.db с предварительным checkpoint WAL.
- PostgreSQL: pg_dump по расписанию.
- Хранение не менее 7 ежедневных копий.
- Проверка восстановления на тестовой БД.

5. Архитектура

5.1. Общая схема

Браузер взаимодействует с FastAPI. FastAPI отдаёт Jinja2-шаблоны и JSON API, работает с SQLAlchemy и БД, вызывает Google API, Robokassa, AITunnel, SMTP и Telegram. APScheduler выполняет фоновые задачи: обработку outbox-уведомлений, проверку дедлайнов и запуск отчётов по расписанию.

Компонент	Назначение
FastAPI + Uvicorn	HTTP-сервер, REST API, серверный рендеринг, зависимости и права.
Jinja2 + Alpine.js + JS	Пользовательский интерфейс, таблицы, формы, модальные окна и интерактивные элементы.
SQLAlchemy 2.0	Единый слой БД для SQLite и PostgreSQL, ORM-модели, транзакции.
APScheduler + croniter	Напоминания, outbox-уведомления, расписание отчётов.
Google API	Drive, Docs, Sheets; сервисный аккаунт и OAuth2 пользователя.
Robokassa	Генерация ссылок и проверка ResultURL.
AITunnel	AI-резюме отчётов с резервным текстом.
Docker Compose	Запуск, монтирование кода, hot reload, PostgreSQL-профиль.

5.2. Стек

- Python 3.12, FastAPI, Uvicorn, SQLAlchemy 2.0, Pydantic 2, pydantic-settings.
- SQLite (по умолчанию) и PostgreSQL 16 (опционально).
- Jinja2, Alpine.js, Chart.js, минимальный JavaScript, собственный CSS в палитре Aillix.
- pwdlib[argon2], itsdangerous, python-multipart, email-validator.
- google-api-python-client, google-auth, google-auth-oauthlib.
- openai (AITunnel), httpx, smtplib, apscheduler, croniter.
- pytest, ruff.

5.3. Структура проекта

app/main.py — точка входа; app/config.py — настройки; app/database.py — движок и сессии; app/models.py — ORM-модели; app/schemas.py — Pydantic-схемы; app/api/ — JSON API; app/web/ — UI-маршруты и шаблоны; app/services/ — бизнес-логика; app/google/ — интеграции Google; tests/ — автотесты; data/ — БД; secrets/ — ключи; docs/ — документация; Dockerfile, docker-compose.yml, start.bat.

5.4. Потоки данных

1. Пользователь входит, получает сессию и активный проект.
2. Форма UI проходит CSRF-проверку и вызывает сервис.
3. Сервис проверяет права, выполняет операцию в транзакции и пишет журнал.
4. При событии создаётся запись в outbox; фоновый обработчик отправляет email/Telegram.
5. Отчёт собирает агрегаты, вызывает AITunnel, пишет Google Sheet и сохраняет ссылку.
6. Планировщик проверяет расписания и дедлайны, создавая новые задания.

5.5. Аутентификация и права

Сессия хранится в подписанной cookie. API принимает Bearer-токен. Проверка прав выполняется зависимостями FastAPI: require_user, require_admin, require_project_access. Администратор имеет доступ ко всем проектам; остальные — только к проектам, где состоят участниками.

5.6. Фоновые задачи

APScheduler запускается в lifespan приложения при SCHEDULER_ENABLED=true. Задачи открывают собственную сессию БД, обрабатывают outbox, дедлайны и расписания. Ошибки логируются и не останавливают приложение. В тестах планировщик отключён.

6. Модель данных

6.1. Общие принципы

- Первичные ключи — целые автоинкрементные.
- Денежные поля — NUMERIC(12,2), валюта — строка ISO 4217.
- Даты — timezone-aware; API отдаёт ISO 8601.
- Мягкое удаление и архивирование вместо физического удаления там, где важна история.
- Внешние ключи с ON DELETE RESTRICT/SET NULL; каскад только для зависимых строк.
- Поле search_text хранит нормализованную строку для поиска без учёта регистра.

6.2. Основные таблицы

Таблица	Ключевые поля	Назначение
users	id, email, password_hash, full_name, role, is_active, telegram_chat_id, notify_email, notify_telegram, created_at	Пользователи и настройки уведомлений
api_tokens	id, user_id, name, token_hash, expires_at, last_used_at	Bearer-токены API
projects	id, name, code, description, status, currency, owner_id, is_archived, created_at	Проекты
project_members	project_id, user_id, role	Участники проектов
clients	id, project_id, name, company, email, phone, status, source, notes, owner_id, is_archived, search_text	Клиенты
deals	id, project_id, client_id NULL, kind, title, amount, currency, stage, description, owner_id, fulfilled_at, search_text	Сделки и заказы
order_items	id, deal_id, product_id NULL, name, quantity, unit_price, total	Позиции заказа

Таблица	Ключевые поля	Назначение
tasks	id, project_id, client_id NULL, deal_id NULL, title, description, due_at, priority, assigned_to_id, is_completed, completed_at, search_text	Задачи
payments	id, project_id, client_id, deal_id, amount, currency, status, method, inv_id, robokassa_link, paid_at, created_at	Платежи
products	id, project_id, sku, name, unit, price, cost, stock_qty, min_stock, is_active	Товары
stock_movements	id, project_id, product_id, movement_type, quantity, unit_cost, reference, comment, created_by_id, created_at	Движения склада
transactions	id, project_id, type, category, amount, currency, transaction_date, description, payment_id NULL, deal_id NULL, created_by_id	Доходы и расходы
reports	id, project_id, report_type, period_from, period_to, spreadsheet_url, summary, created_by_id, created_at	История отчётов
report_schedules	id, project_id, report_type, cron_expression, recipients_email, recipients_telegram, channels, is_active, last_run_at, next_run_at, last_error, created_by_id	Расписание отчётов
notifications	id, user_id NULL, project_id NULL, channel, recipient, subject, body, status, attempts, error, dedupe_key UNIQUE, created_at, sent_at	Outbox уведомлений
google_files	id, entity_type, entity_id, file_id, name, mime_type, web_url, created_at	Ссылки на Google-файлы
activity_log	id, project_id, user_id, entity_type, entity_id, action, details, created_at	Журнал действий
app_settings	key UNIQUE, value, updated_at	Настройки, включая folder_id

6.3. Статусы и этапы

Сущность	Значения
Клиент	new, active, inactive, archived
Сделка	new, negotiation, proposal, won, lost, fulfilled
Задача	open, done (is_completed)
Приоритет	low, normal, high, urgent
Платёж	pending, paid, failed, refunded, canceled
Складская операция	in, out, adjust
Транзакция	income, expense
Роль	admin, manager, viewer

6.4. Индексы и ограничения

- Уникальные: users.email, projects.code, api_tokens.token_hash, notifications.dedupe_key.
- Индексы: clients(project_id, status), deals(project_id, stage), tasks(project_id, due_at, is_completed), payments(project_id, status), products(project_id, sku), transactions(project_id, transaction_date).
- Проверки: положительные суммы, неотрицательный остаток, допустимые статусы.
- Поиск: search_text + LIKE; для PostgreSQL допускается ILIKE.

7. API

7.1. Общие принципы

- Базовый путь /api; формат JSON; OpenAPI на /docs.
- Пагинация: page, per_page (по умолчанию 20, максимум 100).
- Сортировка: sort_by, sort_dir=asc|desc.
- Фильтры: project_id, q, status, stage, assigned_to, date_from, date_to.
- Ошибки: {detail, code, fields} с корректным HTTP-кодом.

7.2. Аутентификация

- POST /api/auth/login — вход, установка сессии.
- POST /api/auth/logout — выход.
- GET /api/auth/me — текущий пользователь и доступные проекты.
- POST /api/auth/tokens — создать API-токен.
- DELETE /api/auth/tokens/{id} — отозвать токен.
- Bearer-токен передаётся в Authorization: Bearer <token>.

7.3. Основные endpoints

Группа	Endpoints
Проекты	GET/POST /api/projects; GET/PUT/DELETE /api/projects/{id}; GET/POST/DELETE /api/projects/{id}/members
Клиенты	GET/POST /api/clients; GET/PUT/DELETE /api/clients/{id}; POST /api/clients/{id}/archive; POST /api/clients/{id}/restore
Сделки	GET/POST /api/deals; GET/PUT/DELETE /api/deals/{id}; POST /api/deals/{id}/items; DELETE /api/deals/{id}/items/{item_id}; POST /api/deals/{id}/fulfill
Задачи	GET/POST /api/tasks; GET/PUT/DELETE /api/tasks/{id}; POST /api/tasks/{id}/complete; POST /api/tasks/{id}/reopen
Платежи	GET/POST /api/payments; GET/PUT /api/payments/{id}; POST /api/payments/{id}/mark-paid; POST /api/payments/{id}/refund; GET /api/payments/{id}/link
Склад	GET/POST /api/products; GET/PUT/DELETE /api/products/{id}; GET/POST /api/stock/movements; GET /api/stock/low
Учёт	GET/POST /api/transactions; GET/PUT/DELETE /api/transactions/{id}; GET /api/accounting/summary
Отчёты	POST /api/reports/generate; GET /api/reports; GET /api/reports/{id}; POST /api/reports/{id}/retry
Расписания	GET/POST /api/report-schedules; GET/PUT/DELETE /api/report-schedules/{id}; POST /api/report-schedules/{id}/run
Уведомления	GET /api/notifications; POST /api/notifications/test; GET /api/notifications/status
Google	GET /api/google/status; GET /api/google/files; POST /api/google/docs; POST /api/google/sheets; POST /api/google/bootstrap
Поиск	GET /api/search?q=
Админ	GET/POST /api/users; GET/PUT/DELETE /api/users/{id}; GET /api/activity
Система	GET /health; GET /api/system/info

8. Развёртывание

8.1. Варианты запуска

- SQLite: docker compose up -d --build; база в data/crm.db; подходит для одного пользователя и локальной работы.

• PostgreSQL: docker compose -f docker-compose.yml -f docker-compose.postgres.yml up -d --build; DATABASE_URL указывает на сервис db.

• Локальный Python: .venv + pip install -r requirements.txt + uvicorn.

• start.bat автоматизирует остановку старых процессов, очистку кэша, сборку, запуск, healthcheck и открытие браузера.

8.2. Docker и hot reload

• Код монтируется в /app.

• Uvicorn запускается с --reload и --reload-dir /app.

• WATCHFILES_FORCE_POLLING=true для Windows bind mounts.

• Изменения Python перезапускают сервер; шаблоны и CSS обновляются без пересборки.

• Порт 8000 привязан к 127.0.0.1.

8.3. Переменные окружения

Основные переменные: APP_ENV, SECRET_KEY, DATABASE_URL, ADMIN_EMAIL, ADMIN_PASSWORD, GOOGLE_SERVICE_ACCOUNT_FILE, GOOGLE_OAUTH_CLIENT_SECRET_FILE, GOOGLE_OAUTH_TOKEN_FILE, GOOGLE_DRIVE_FOLDER_ID, REPORT_SPREADSHEET_ID, ROBOKASSA_MERCHANT_LOGIN, ROBOKASSA_PASSWORD1, ROBOKASSA_PASSWORD2, ROBOKASSA_TEST_MODE, SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, TELEGRAM_BOT_TOKEN, AITUNNEL_API_KEY, SCHEDULER_ENABLED, TIMEZONE. Полный список — в .env.example.

8.4. Google Cloud

1. Включить Google Drive API, Google Docs API, Google Sheets API.
2. Создать сервисный аккаунт и скачать JSON-ключ.
3. Создать OAuth-клиент типа Desktop и скачать client_secret JSON.
4. Добавить redirect URI http://localhost:8765/.
5. Добавить тестового пользователя в OAuth consent screen.
6. Выдать сервисному аккаунту доступ Editor к папке и таблице отчётов.
7. Выполнить bootstrap папки и вход OAuth2.

8.5. Robokassa

• Тестовый режим: ROBOKASSA_TEST_MODE=true, IsTest=1.

• Пароль 1 — для ссылки, пароль 2 — для ResultURL.

• ResultURL должен быть доступен из интернета; локально используйте туннель или ручную отметку.

• При отсутствии реквизитов кнопка ссылки отключена, ручная оплата доступна.

8.6. Уведомления

• SMTP: host, port, user, password, TLS, from.

• Telegram: bot token и chat_id пользователя.

• При отсутствии настроек уведомление помечается skipped, приложение продолжает работу.

• Тестовая отправка доступна в настройках.

8.7. Резервное копирование

• SQLite: scripts/backup_db.py создаёт копию с checkpoint WAL.

• PostgreSQL: pg_dump через docker compose exec db.

• Копии хранятся в data/backups с датой и временем.

• Восстановление описано в docs/admin-guide.txt.

9. Тестирование и приёмка

9.1. Виды тестов

• Модульные: БД, поиск, права, платежи, склад, учёт, уведомления.
• Интеграционные: API через TestClient, сессии, CSRF, роли.
• Моки: Google API, SMTP, Telegram, AITunnel.
• Ручные: UI, Docker, hot reload, OAuth2, Robokassa-ссылка.

9.2. Чек-лист приёмки

1. start.bat останавливает старые процессы, чистит кэш, собирает и запускает приложение.
2. Приложение доступно на http://127.0.0.1:8000, /health отвечает 200.
3. Администратор входит, создаёт проект, пользователя и участника проекта.
4. Менеджер видит только свои проекты и не может открыть чужой проект.
5. Клиент, сделка, заказ, задача и платёж создаются, редактируются и удаляются согласно правам.
6. Поиск по русским именам работает без учёта регистра.
7. Заказ с позициями списывает товары со склада и создаёт движения.
8. Оплата платежа создаёт доходную транзакцию; возврат создаёт расходную.
9. Robokassa-ссылка формируется с корректной подписью; ResultURL проверяет подпись.
10. Отчёт генерируется в Google Таблицу, AI-резюме или резервный текст присутствует.
11. Расписание отчёта запускается вручную и по cron, создаёт уведомления.
12. Email и Telegram-уведомления работают при заданных настройках; без них — skipped.
13. Google Drive показывает файлы папки; OAuth2 создаёт Документ и Таблицу от имени пользователя.
14. CSV открывается в Excel без искажения кодировки.
15. Секреты отсутствуют в git; README без Markdown соответствует фактическим командам.
16. pytest проходит; Docker-образ собирается; финальный commit/push выполнен.

9.3. Definition of Done

• Все функции раздела 3 реализованы и проверены.
• Критичные ошибки отсутствуют; известные ограничения описаны.
• README, ТЗ, Google-инструкция и руководство администратора обновлены.
• Миграции/инициализация БД воспроизводимы на чистой машине.
• Секреты не попадают в репозиторий.
• Финальный commit и push выполнены.

10. Этапы и сроки

Этап	Содержание	Оценка
0	Каркас, .env, secrets, Docker, git, README, ТЗ	4–5 ч
1	Google Drive сервисного аккаунта + CLI	3–4 ч
2	OAuth2, Docs/Sheets, bootstrap папки	5–6 ч
3	SQLAlchemy-модели, БД, миграции, seed, тесты	5–6 ч
4	Auth, пользователи, роли, проекты, API	6–8 ч
5	Клиенты, сделки, задачи, поиск, API	6–8 ч

Этап	Содержание	Оценка
6	Платежи, Robokassa, склад, учёт	6–8 ч
7	Отчёты, Google Sheets, AI, расписания, уведомления	8–10 ч
8	UI, палитра Aillix, дашборд, формы	10–12 ч
9	Docker, start.bat, тесты, README, git	6–8 ч

Итого: 59–75 часов. Ориентировочный срок — 8–10 рабочих дней при последовательной реализации. Возможна поэтапная сдача: сначала ядро CRM, затем Google/Robokassa, затем отчёты и уведомления.

11. Ограничения и допущения

- Заказчик предоставляет доступы к Google Cloud, Robokassa, SMTP и Telegram.
- Первичная настройка OAuth2 требует ручного подтверждения в браузере.
- Внешние API могут быть недоступны; система должна деградировать без потери данных.
- Юридическая и налоговая отчётность не формируется.
- При использовании PostgreSQL требуется отдельный сервер или контейнер.
- Планировщик не гарантирует запуск при остановленном приложении.
- Дизайн следует палитре Aillix, но не копирует логотипы и фирменные материалы сайта.

12. Риски

Риск	Вероятность	Митигация
OAuth2 redirect и токен в Docker	Средняя	Проброс 8765, инструкция, повторный вход, Production-режим OAuth
Сервисный аккаунт без квоты Drive	Высокая	Создание файлов через OAuth2, сервисный аккаунт — чтение и работа в расшаренной папке
Нет доступа к таблице/папке	Средняя	Проверка прав Editor, bootstrap, диагностика 403
Windows hot reload не видит файлы	Средняя	WATCHFILES_FORCE_POLLING=true, polling
SQLite на bind mount	Средняя	WAL, busy_timeout, named volume при проблемах
Секреты в git	Высокая	.gitignore, проверка перед commit, секреты только в secrets/.env
Сложность полного функционала	Высокая	Поэтапная сдача, тесты, приоритет MVP
Сбои SMTP/Telegram	Средняя	Outbox, повторные попытки, статус skipped
Дублирование планировщика	Низкая	Один экземпляр приложения, документация

13. Приложения

13.1. Переменные окружения

Переменная	Назначение
SECRET_KEY	Подпись сессий и CSRF
DATABASE_URL	sqlite:///./data/crm.db или postgresql+psycopg://...
ADMIN_EMAIL / ADMIN_PASSWORD	Первичный администратор
GOOGLE_SERVICE_ACCOUNT_FILE	Путь к JSON сервисного аккаунта
GOOGLE_OAUTH_CLIENT_SECRET_FILE	Путь к client_secret JSON
GOOGLE_OAUTH_TOKEN_FILE	Путь к token.json
GOOGLE_DRIVE_FOLDER_ID	Папка Google Drive

Переменная	Назначение
REPORT_SPREADSHEET_ID	Рабочая Google Таблица отчётов
ROBOKASSA_MERCHANT_LOGIN / PASSWORD1 / PASSWORD2	Реквизиты Robokassa
ROBOKASSA_TEST_MODE	true/false
SMTP_HOST / PORT / USER / PASSWORD / FROM	Почта
TELEGRAM_BOT_TOKEN	Telegram Bot API
AITUNNEL_API_KEY / BASE_URL / MODEL	AI-резюме
SCHEDULER_ENABLED	Включение фоновых задач
TIMEZONE	Europe/Moscow

13.2. Примеры cron

- Ежедневно в 09:00: 0 9 * * *
- По понедельникам в 10:00: 0 10 * * 1
- Первое число месяца в 08:30: 30 8 1 * *
- Каждые 6 часов: 0 */6 * * *

13.3. Репозиторий

https://github.com/markmus11012020-max/custom-CRM.git

Документ считается утверждённым после подтверждения заказчиком. Изменения объёма оформляются новой версией ТЗ.